

Prädikate sind Aussagen über mathematische Objekte, die wahr oder falsch sein können. «Funktionale» Prädikate mit Beispielen Rückgabewerten: $P, Q(n), R(x, y, z)$.

$A \Rightarrow B \Leftrightarrow (\neg A \vee B) \Leftrightarrow (A \wedge B) \vee (\neg A \wedge \neg B) \Leftrightarrow A \vee \neg B$
 $A \Rightarrow B \Leftrightarrow (\neg A \wedge B) \Leftrightarrow A \wedge \neg(B \Rightarrow C) \Leftrightarrow (A \Rightarrow B) \wedge (B \Rightarrow C)$

Begriff	Bedeutung
Abtrennungsregel	$(A \wedge (A \Rightarrow B)) \Rightarrow B$
Kommutativität	$(A \wedge B) \Leftrightarrow (B \wedge A)$ $(A \vee B) \Leftrightarrow (B \vee A)$
Assoziativität	$A \wedge (B \wedge C) \Leftrightarrow (A \wedge B) \wedge C$ $A \vee (B \vee C) \Leftrightarrow (A \vee B) \vee C$
Distributivität	$A \wedge (B \vee C) \Leftrightarrow (A \wedge B) \vee (A \wedge C)$ $A \vee (B \wedge C) \Leftrightarrow (A \vee B) \wedge (A \vee C)$
Absorption	$A \vee (A \wedge B) \Leftrightarrow A$ $A \wedge (A \vee B) \Leftrightarrow A$
Idempotenz	$A \vee A = A$ $A \wedge A = A$
Doppelte Negation	$\neg(\neg A) \Leftrightarrow \neg\neg A \Leftrightarrow A$
Konstanten	$W = \text{wahr}$ $F = \text{falsch}$
de Morgan	$\neg(A \wedge B) \Leftrightarrow \neg A \vee \neg B$ $\neg(A \vee B) \Leftrightarrow \neg A \wedge \neg B$

NORMALFORMEN

Normalformen (allgemein, **kanonische Formen**) helfen, das Vergleichsproblem zu lösen (ob zwei Aussagen dieselben sind).

$$P_1 \wedge \dots \wedge P_n = \forall i \in \{1, \dots, n\} (P_i) = \bigwedge_{i=1}^n P_i$$
$$P_1 \vee \dots \vee P_n = \exists i \in \{1, \dots, n\} (P_i) = \bigvee_{i=1}^n P_i$$

NEGATION

Nicht für alle = Es gibt einen Fall, für den nicht
 $\neg \forall i \in \{1, \dots, n\} (P_i) \Leftrightarrow \neg (P_1 \wedge \dots \wedge P_n) \Leftrightarrow \neg P_1 \vee \dots \vee \neg P_n \Leftrightarrow \exists i \in \{1, \dots, n\} \neg P_i$

MENGEN

KONSTRUKTION

Grundmenge G , Prädikat $P(x)$. Konstruierte Menge $A = \{x \in G \mid P(x)\}$

OPERATIONEN

Begriff	Bedeutung
Vereinigung	$A \cup B = \{x \mid x \in A \vee x \in B\}$
Schnittmenge	$A \cap B = \{x \mid x \in A \wedge x \in B\}$
Komplement	$\bar{A} = \{x \mid x \notin A\}$
Differenz	$A \setminus B = \{x \in A \mid x \notin B\}$

TUPEL

$$A \times B = \{(a, b) \mid a \in A \wedge b \in B\}$$

n -Tupel:

$$\bigtimes_{i=1}^n A_i = \{(a_1, a_2, \dots, a_n) \mid a_i \in A_i\}$$

BEWEISE

Eine Folge von logischen Schlüssen, die zeigen, dass die Aussage aus den gegebenen Voraussetzungen folgt.

BEWEISTECHNIK

Beweistechnik	Anwendungsfall
Konstruktiver Beweis	Liefert explizite «Lösung» / Algorithmus
Widerspruchsbeweis	Geeignet für Unmöglichkeitss Aussagen. Z.B. $\sqrt{2} \notin \mathbb{Q}$
Vollständige Induktion	Für Aussagen, die für alle $n \in \mathbb{N}$ gelten.

SPRACHEN

Regulär	Kontextfrei	Turing-erkennbar
DEA NEA Regex	CFG PDA Palindrome {0^n 1^n n ≥ 0}	Primzahlen {a^n b^n c^n n ≥ 0}

Alle Sprachen sind überabzählbar

Begriff	Bedeutung
Alphabet	Eine nichtleere endliche Menge Σ heisst Alphabet . Die Elemente von Σ heissen Zeichen .
Wort	Eine Zeichenkette der Länge n ist ein n -Tupel in $\Sigma^n = \Sigma \times \dots \times \Sigma$. Ein Element von Σ^n heisst Wort der Länge n . Wörter haben immer endliche Länge.
Leeres Wort	Die Zeichenkette $\epsilon \in \Sigma^0 = \{\epsilon\}$ der Länge 0 heisst das leere Wort .
Menge aller Wörter	Leeres Wort ist immer drin $\Sigma^* = \{\epsilon\} \cup \Sigma \cup \Sigma^2 \cup \Sigma^3 \cup \dots = \bigcup_{k=0}^{\infty} \Sigma^k$
Abgekürzte Schreibweisen von Wörtern	$\langle A, u, t, o, S, p, r \rangle = \text{AutoSpr}$ $a^3 b^5 = \text{aaabbbbbb}$ $b^5 a^3 = \text{bbbbbaaa}$
Sprache	Teilmenge $L \subset \Sigma^*$

WORTLÄNGE

Begriff	Bedeutung
Länge des Wortes	$w \in \Sigma^n \Rightarrow w = n$, n ist Länge des Wortes w
Anzahl Zeichen	Sei $w \in \Sigma^n, a \in \Sigma$. Dann ist $ w _a$ die Anzahl Zeichen a im Wort w

Beispiel 1: Wortlänge

$$|\epsilon| = 0 \quad |01010|_1 = 2 \quad |a^3 b^5| = 8$$
$$|01010|_0 = 3 \quad |01010|_3 = 0 \quad |(1291)^7| = 7|1291| = 7 \cdot 4 = 28$$

Beispiel 2: Sprache

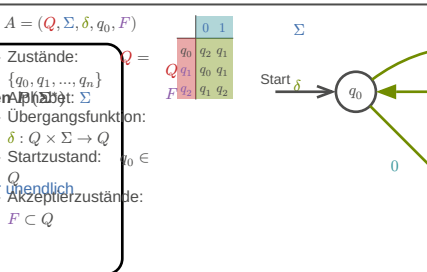
$L = \Sigma^* \subset \Sigma^*$
 $L = \emptyset \subset \Sigma^*$, die leere Sprache
 $\Sigma = \{0, 1\}$, Sprache aller Binärstrings: $L = \Sigma^*$
Unicode, $J = \{w \in \Sigma^* \mid w \text{ wird vom Java-Compiler akzeptiert}\}$
eine "Maschine", $L(M) = \{w \in \Sigma^* \mid w \text{ wird von } M \text{ akzeptiert}\}$

Beobachtungen 1

- $|w^n| = n \cdot |w|$
- Zu jeder Maschine M gibt es eine Sprache $L(M)$

DETERMINISTISCHE ENDLICHE AUTOMATEN (DEA)

Definition: Tabellenform, Zustandsdiagramm von A



Wichtig: Ein Pfeil für jedes Zeichen in jedem Zustand für validen DEA

WÖRTER EINER REGULÄREN SPRACHE

ÜBERGANGSFUNKTIONEN FÜR WÖRTER

$\delta: Q \times \Sigma^* \rightarrow Q: (q, w = a_1, \dots, a_n) \mapsto \delta(\dots \delta(\delta(q, a_1), a_2), \dots, a_n)$

Übergänge ausgehend von q für Zeichen $a_1, \dots, a_n$ nacheinander anwenden.

WORT AKZEPTIEREN

Der DEA $A = (\Sigma, Q, q_0, \delta, F)$ **akzeptiert** das Wort $w \in \Sigma^*$, wenn er A von Startzustand in einen Akzeptierzustand $\delta(q_0, w) \in F$ überführt.

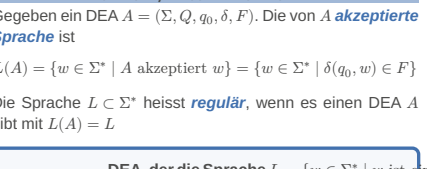
AKZEPTIERTE SPRACHE, REGULÄRE SPRACHE

Gegeben ein DEA $A = (\Sigma, Q, q_0, \delta, F)$. Die von A **akzeptierte Sprache** ist

$$L(A) = \{w \in \Sigma^* \mid A \text{ akzeptiert } w\} = \{w \in \Sigma^* \mid \delta(q_0, w) \in F\}$$

Die Sprache $L \subset \Sigma^*$ heisst **regulär**, wenn es einen DEA A gibt mit $L(A) = L$

Beispiel 3: DEA, der die Sprache $L = \{w \in \Sigma^* \mid w \text{ ist eine ungerade Zahl im Dreiersystem}\}$ akzeptiert



MYHILL-NERODE AUTOMAT

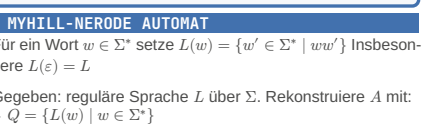
Für ein Wort $w \in \Sigma^*$ setze $L(w) = \{w' \in \Sigma^* \mid ww' \in L\}$. Insbesondere $L(\epsilon) = L$

Gegeben: reguläre Sprache L über Σ . Rekonstruiere A mit:

- $Q = \{L(w) \mid w \in \Sigma^*\}$
- $q_0 = L(\epsilon) = L$
- $F = \{L(w) \in Q \mid \epsilon \in L(w)\}$
- $\delta(L(w), a) = L(wa)$

Gleicher Zustand $\Leftrightarrow$ gleiches $L(w)$

Beispiel 4: $\Sigma = \{0, 1\}, L = \{w \in \Sigma^* \mid |w|_0 \text{ gerade}\}$



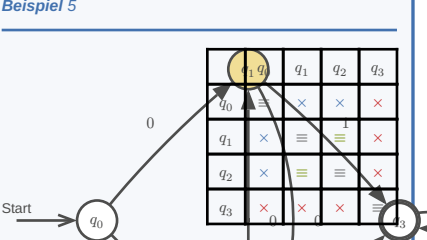
MINIMALAUTOMAT

Ziel: Nicht unterscheidbare Zustände zusammenlegen

Vorgehen:

- Akzeptierzustand $\neq$ Nichtakzeptierzustand
- Iteration: alle unterscheidbaren Paare suchen
- Verbleibende Paare sind ununterscheidbar $\rightarrow$ zusammenlegen

Beispiel 5



Algorithmus

- $q \in F, q \notin F$
- Unterscheidbar nach Übergang
- Iteration bis keine Änderung mehr $\Rightarrow q_1$ und q_2 sind nicht unterscheidbar

PUMPING LEMMA

Ist L eine reguläre Sprache, dann gibt es $N \in \mathbb{N}$, die pumping length so, dass jedes Wort $w \in L$ mit $|w| \geq N$ in drei Teile $w = xyz$ zerlegt werden kann mit:

- $|xy| \leq N$
- $|y| > 0$
- $xy^kz \in L \forall k \in \mathbb{N}$

oder: genügend lange Wörter ($|w| \geq N$) einer regulären Sprache ($w \in L$) können alle in einem Anfangsstück der Länge N ($|xy| \leq N$) aufgepumpt werden ($xy^kz \in L$).

Was zeichnet reguläre Sprachen aus?

- Reguläre Ausdrücke
- Lange Wörter: * kommt im regulären Ausdruck vor
- Blöcke mit * können beliebig oft wiederholt werden $\Rightarrow$ Wörter können «aufgepumpt» werden

BEWEIS

Behauptung: Die Sprache $L \subset \Sigma^*$ ist nicht regulär.

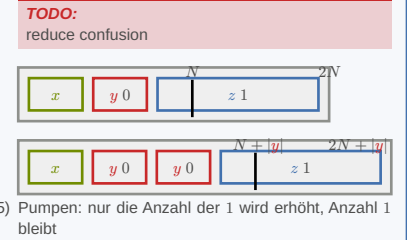
Beweis (Widerspruch):

- Annahme: L ist regulär
- Gemäss Pumping Lemma gibt es die Pumping Length N
- Es darf keine Annahme über die konkrete Grösse von N gemacht werden!
- Wähle ein Wort $w \in L$ mit $|w| \geq N$
- Die Definition **muss** N verwenden!
- Aufteilung des Wortes gemäss Pumping Lemma $w = xyz, |xy| \leq N, |y| > 0$
- Auswirkung des Pumpens $\Rightarrow xy^kz \notin L$ für mindestens ein $k \in \mathbb{N}$ (mit Begründung!)
- Widerspruch und Schlussfolgerung, dass die Annahme nicht zutreffen kann

Beispiel 6: $L = \{0^n 1^n \mid n \geq 0\}$ ist nicht regulär

- Annahme: L ist regulär
- $\exists N \in \mathbb{N}$, Pumping Length
- $w = 0^N 1^N$
- Unterteilung $w = xyz$

TODO: reduce confusion



- Pumpen: nur die Anzahl der 1 wird erhöht, Anzahl 1 bleibt
- $xy^kz \notin L$ für $k \neq 1$, im Widerspruch zum Pumping Lemma

NICHTDETERMINISTISCHE ENDLICHE AUTOMATEN (NEA)

Definition

$A = (Q, \Sigma, \delta, q_0, F)$

- Endliche Menge von Zuständen: Q
- Alphabet: Σ
- Übergangsfunktion: $\delta: Q \times \Sigma \rightarrow P(Q)$
- Startzustand: $q_0 \in Q$
- Akzeptierzustände: $F \subset Q$

Akzeptieren

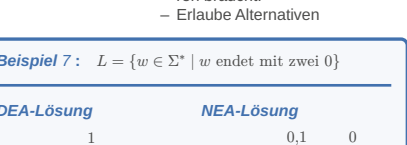
Ein NEA A **akzeptiert** das Wort $w \in \Sigma^*$, wenn es eine **Wahl** von Übergängen gibt, derart, dass das Wort w den Automaten in einen Akzeptierzustand überführt.

Faustregeln

- Nur genau diejenigen Pfeile einzeichnen, die man zum Akzeptieren braucht.
- Erlaube Alternativen

Beispiel 7: $L = \{w \in \Sigma^* \mid w \text{ endet mit zwei 0}\}$

DEA-Lösung NEA-Lösung



UMGANG MIT MEHRDEUTIGEN ÜBERGÄNGEN

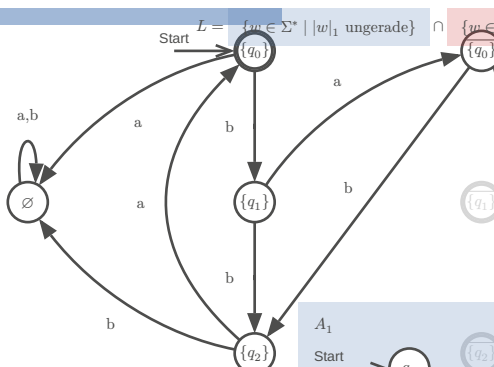
a-Übergang von q_1 aus nicht eindeutig

Welchen Übergang soll man nehmen?

- Beide Möglichkeiten probieren und akzeptieren, falls eine der Möglichkeiten auf einen Akzeptierzustand führt.
- Zufallsgenerator (probabilistischer endlicher Automat, PEA)



- Zustand = Menge der möglichen Zustände
- Akzeptieren = Ob NEA im Zustand sein könnte



Gegeben $\delta : Q \times \Sigma \rightarrow P(Q)$ eines NEA. Übergangsfunktion
für Mengen $M \subset Q$

$$\delta' : P(Q) \times \Sigma \rightarrow P(Q) : (M, a) \mapsto \delta'(M, a) = \bigcup_{q \in M} \delta(q, a)$$

Implementation

Ein NEA A kann in einen DEA A' umgewandelt werden mit

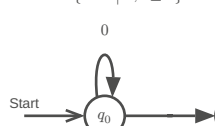
- $Q' = P(Q)$
- $\Sigma' = \Sigma$
- δ' wie oben definiert
- $q'_0 = \{q_0\}$
- $F' = \{M \in P(Q) \mid F \cap M \neq \emptyset\}$

- Zustand $M \subset Q$ realisiert durch Markierung der Zustände in M
- δ' realisiert durch Verschieben von Markierungen
- Akzeptieren: mindestens ein Akzeptierzustand markiert

NEA _ε	
Begriff	Bedeutung
ε-Übergang	Kann ohne Verarbeitung eines Zeichens genommen werden. $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \rightarrow P(Q)$
NEA _ε	NEA mit ε-Übergängen

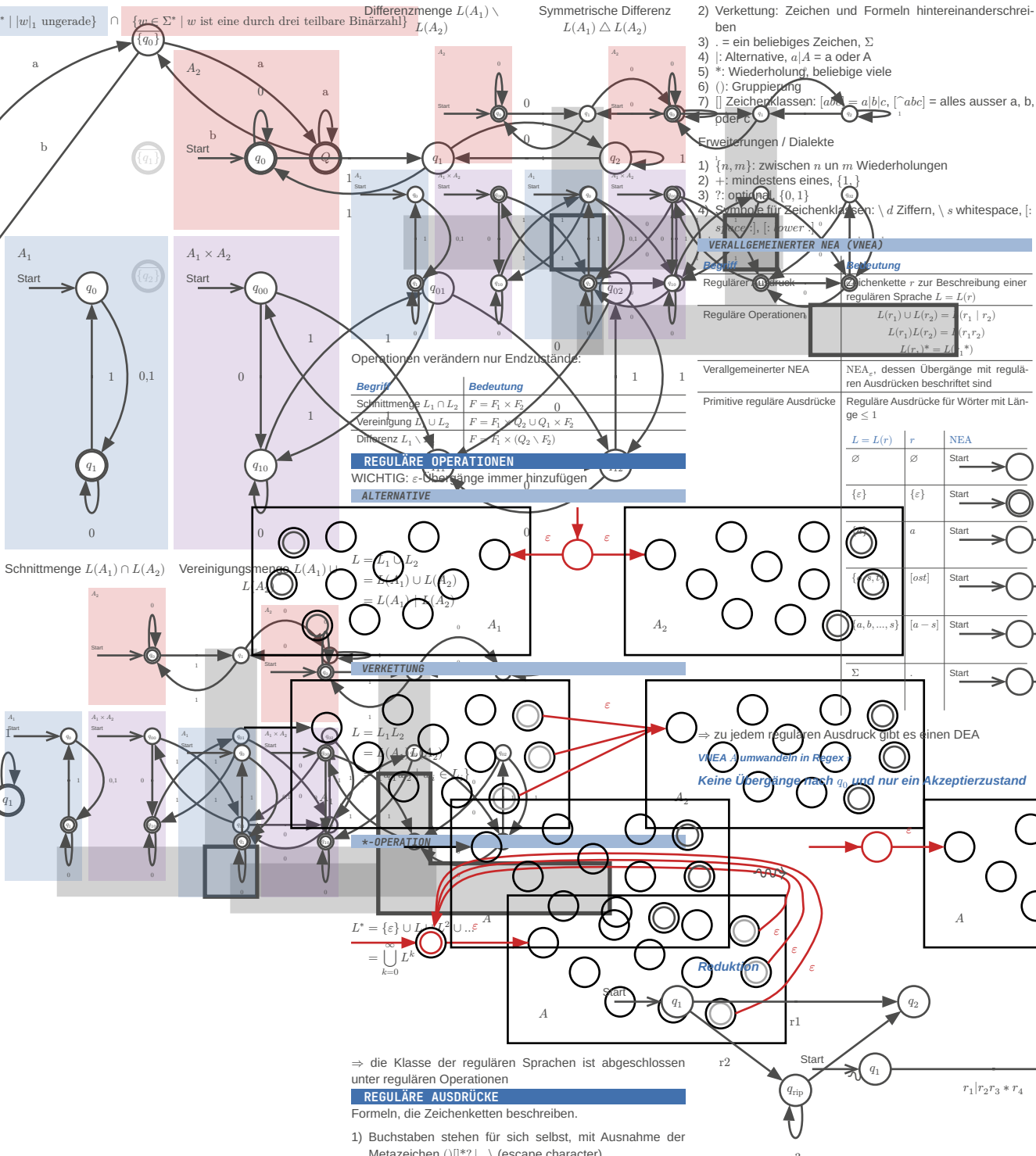
Einen NEA_{ε} kann man immer
in einen NEA umwandeln.
Beweis:

$$L = \{0^k 1^l \mid k, l \geq 0\}$$

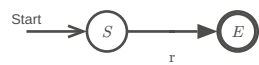


Lassen reguläre Sprachen regulär sein.

Produktautomat: $L_i = L(A_i)$



Nach Entfernen aller Zwischenzustände $q_{\text{rip}} \in Q$ bleibt ein regulärer Ausdruck r von A



⇒ jede reguläre Sprache lässt sich mit einem regulären Ausdruck beschreiben $\{L(A) \mid A \text{ ein DEA}\} = \{L(r) \mid r \text{ ein regulärer Ausdruck}\}$

Interessantes projekt (DEA Lexer DSL): [Ragel](#)

TESTSTRATEGIE	
Messung	Folgerung
Für $n = 1, 2, 3, \dots$ - Regulären Ausdruck erzeugen $r = \underbrace{a^1? a^2? a^3? \dots a^{n-1} a^n a^{n-1} \dots a^3 a^2 a^1}_{n-1}$ - Laufzeit für Akzeptieren von a^n durch r messen	- Laufzeit $O(2^n)$: NEA-Implementa-tion - Laufzeit $O(n)$: DEA-Implementa-tion

KONTEXTFREIE SPRACHEN

<i>Begriff</i>	<i>Bedeutung</i>
CFL	Context Free Language
CFG	Context Free Grammar
PDA	Pushdown Automata

KONTEXTFREIE GRAMMATIK UND SPRACHE

Kontextfreie Grammatik: Parse Tree

$$L = \{w \in \{0, 1\}^* \mid |w|_0 = |w|_1\}$$

- V : Variablen
- Σ : Terminalsymbole (Alphabet)
- R : Regeln der Form $A \rightarrow x_1 x_2 \dots x_n$ mit $A \in V, x_i \in V \cup \Sigma$
- S_0 Startvariable

Ableitung und erzeugte Sprache

- Regel $A \rightarrow uv$ erzeugt aus uAv das Wort $uwv : uAv =$

– v aus u ableiten: $u \Rightarrow v$

$u_1 \Rightarrow u_2 \Rightarrow \dots \Rightarrow u_n \Rightarrow v$
 oder $u \Rightarrow v$
 $L(G) = \{w \in \Sigma^* \mid G \xRightarrow{*} w\}$

- $L(G) \equiv \{w \in \Sigma^+ \mid S \Rightarrow w\}$
von G erzeugte kontextfreie Sprache

Erzeugte Sprache: Die Menge der Wörter, die von einem Automaten erzeugt werden.

ner kontextfreien Gramma-
tik G aus den Startvariablen

abgeleitet werden können
wird mit $L(G)$ bezeichnet

und heißt die von G erzeugte Sprache.

Kontextfreie Sprache: Eine Sprache L heißt kontextfrei, wenn es eine kontextfreie Grammatik gibt, die L erzeugt.

freie Grammatik gibt, die die Sprache $L \equiv L(G)$ er

Beispiel 8: $L = \{0^n 1^n \mid n > 0\}$

Grammatik	$L =$ Wörter, die erzeugt werden müssen
$\{0^n 1^n \mid n > 0\}$	

- 1) Variablen: $V = \{S\}$
- 2) Terminalsymbole: $\Sigma = \{0, 1\}$
- 3) Regeln: $R = \{S \rightarrow \varepsilon, 0S1\}$

4) Startvariable: S

Es ist nicht garantiert, dass es für die Ableitung eines Wortes nur einen einzigen Syntaxbaum gibt. Man sagt, die Grammatik ist **nicht eindeutig**, wenn sie mehrere Syntaxbäume für die gleichen Wörter erlaubt.

REGULÄRE OPERATIONEN
Die Klasse der kontextfreien Sprachen ist abgeschlossen unter regulären Operationen.

Reguläre Sprachen sind kontextfrei.

Grammatik für reguläre Operationen

Seien L_1 und L_2 kontextfreie Sprachen mit Grammatiken $G_i = (V_i, \Sigma, R_i, S_i)$. Wir nehmen an, dass die Variablen- und Regelmengen disjunkt sind, also $V_1 \cap V_2 = \emptyset \wedge R_1 \cap R_2 = \emptyset$. Zudem sei S_0 eine Variable, die nicht in $V_1 \cup V_2$ enthalten ist. Dann gilt:

Alternative $L_1 \mid L_2$ ist kontextfrei mit der Grammatik

$G = (V_1 \cup V_2 \cup \{S_0\}, \Sigma, R = R_1 \cup R_2 \cup \{S_0 \rightarrow S_1 \mid S_2\}, S_0)$

Verkettung $L_1 L_2$ ist kontextfrei mit der Grammatik

$G = (V_1 \cup V_2 \cup \{S_0\}, \Sigma, R = R_1 \cup R_2 \cup \{S_0 \rightarrow S_1 S_2\}, S_0)$

***Operation L_1^* ist kontextfrei mit der Grammatik**

$G = (V_1 \cup \{S_0\}, \Sigma, R = R_1 \cup \{S_0 \rightarrow S_0 S_1, S_0 \rightarrow \varepsilon\}, S_0)$

KONTEXTFREI
Regeln ohne Kontext

Es spielt keine Rolle, in welchem Kontext das Zeichen A vorkommt, die Regel kann immer angewendet werden

$S \rightarrow A \mid C$
 $A \rightarrow a$
 $A \rightarrow aAb$
 $C \rightarrow bA$

Regeln mit Kontext

Je nach **Kontext** kann eine Regel nicht unbedingt angewendet werden

$S \rightarrow C \mid aC \mid bC$
 $aC \rightarrow A$
 $bC \rightarrow B$
 $S \rightarrow aC \rightarrow A$
 $S \rightarrow bC \rightarrow B$
 $S \rightarrow C \rightarrow ?$

CHOMSKY-NORMALFORM (CNF)
Eine CFG ist in **Chomsky-Normalform**, wenn S auf der rechten Seite nicht vorkommt und jede Regel von der Form $A \rightarrow BC$ oder $A \rightarrow a$ ist, zusätzlich ist die Regel $S \rightarrow \varepsilon$ erlaubt.

Umwandlung

- Neue Startvariable $S_0 \rightarrow S$ (wenn nötig)
- ε -Regeln: $A \rightarrow \varepsilon \Rightarrow A$ kann weggelassen werden $\Rightarrow$
 $\begin{cases} B \rightarrow AC \\ A \rightarrow \varepsilon \end{cases} \Rightarrow B \rightarrow C$
- Unit-Rules: $A \rightarrow B \Rightarrow$ aus A kann man wie aus B auch CD machen $\Rightarrow$
 $\begin{cases} A \rightarrow B \\ B \rightarrow CD \end{cases} \Rightarrow A \rightarrow CD$
- Verkettungen: $A \rightarrow u_1 u_2 \dots u_n$ ersetzen durch $A \rightarrow u_1 A_1, A_1 \rightarrow u_2 A_2, \dots, A_{n-2} \rightarrow u_{n-1} u_n$ und falls u_i ein Terminalsymbol ist: $A_{i-1} \rightarrow u_i A_i, U_i \rightarrow u_i$.

Folgerungen

- Ableitung eines Wortes $w \in L(G)$ ist immer in $2|w| - 1$ Regelanwendungen möglich. Beweis:
 - $|w| - 1$ Regeln der Form $A \rightarrow BC$ um aus S ein Wort aus $|w|$ Variablen zu erzeugen
 - $|w|$ Regeln der Form $A \rightarrow a$ um das Wort w zu erzeugen $\Rightarrow$ insgesamt $2|w| - 1$ Regelanwendungen
- Deterministischer Parse-Algorithmus mit Laufzeit $O(|w|^3) \rightarrow$

Beispiel 9:
 $S \rightarrow ASA \mid aB$
 $A \rightarrow B \mid S$
 $B \rightarrow b \mid \varepsilon$

1) Startvariable
 $S_0 \rightarrow S$
 $S \rightarrow ASA \mid aB$
 $A \rightarrow B \mid S$
 $B \rightarrow b \mid \varepsilon$

2) ε -Regel
 $S_0 \rightarrow S$
 $S \rightarrow ASA \mid aB \mid a$
 $A \rightarrow B \mid \varepsilon \mid S$
 $B \rightarrow b$

3) Unit-Regel
 $S_0 \rightarrow ASA \mid SA \mid AS \mid aB \mid aS_0 \rightarrow ASA \mid SA \mid AS \mid aB \mid a$
 $S \rightarrow ASA \mid SA \mid AS \mid aB \mid a$
 $A \rightarrow B \mid ASA \mid SA \mid AS \mid aB \mid a$
 $B \rightarrow b$

4) Komplexe Regeln
 $S_0 \rightarrow AA_1 \mid SA \mid AS \mid UB \mid a$
 $S \rightarrow AA_1 \mid SA \mid AS \mid UB \mid a$
 $A \rightarrow b \mid AA_1 \mid SA \mid AS \mid UB \mid a$
 $AA_1 \rightarrow SA$
 $U \rightarrow a$
 $B \rightarrow b$

PARSING
COCKE-YOUNGER-KASAMI ALGORITHMUS (CYK)
Gegeben

- Grammatik $G = (V, \Sigma, R, S)$
- Variable $A \in V$
- Wort $w \in \Sigma^*$

Frage

Ist w ableitbar? Formell geschrieben: $A \xRightarrow{*} w$
– Spezialfall $w = \varepsilon$: $A \xRightarrow{*} \varepsilon \Leftrightarrow A \rightarrow \varepsilon \in R$ (Epsilonregel, $A = S$)
– Spezialfall $|w| = 1$: $A \xRightarrow{*} w \Leftrightarrow A \rightarrow w \in R$ (Terminalsymbolregel)
– Fall $|w| > 1$:

$A \xRightarrow{*} w \Rightarrow \exists \begin{cases} A \rightarrow BC \in R \\ w = w_1 w_2 \\ w_i \in \Sigma^* \end{cases} \text{ mit } \begin{cases} B \xRightarrow{*} w_1 \\ C \xRightarrow{*} w_2 \end{cases}$

ABLEITUNGSDREIECK
Ideen

- Parse Tree aus $A \rightarrow BC$ und $T \rightarrow t$
- Variablen in Tabelle füllen

Prinzip

– Einem Teilwort entspricht ein Feld der Tabelle (rot hinterlegt)
– Das Feld wird mit den Variablen gefüllt, aus denen das Teilwort abgeleitet werden kann.

Beispiel

Parse des Wortes $()()()$
 $AB \mid CD \mid CU \mid SSB \rightarrow$
 SD
 $C \rightarrow$
 $D \rightarrow$

Definitionen

Die Felder (k, l_1) und $(k + l_1, l - l_1)$ heißen **korrespondierende Felder** im Ab-

leitungs-dreieck des Feldes (k, l) .

TODO:
rekursion vs iteration (buch 149)

BACKUS-NAUR-FORM (BNF)
Spezifikation der Regeln in maschinen-lesbarer Form:

- Variablen: **<variablen-name>**
- Einzelne Zeichen: **A**
- Zeichenketten: **'BEISPIEL'**
- Regeln: **<variablen-name> ::= Ausdruck**
- Ausdrücke sind Folgen von Variablen, einzelnen Zeichen oder Zeichenketten, getrennt durch **|**

Beispiel 10: BNF für Expression-Term-Factor Grammatik

Ausgangsgrammatik

expression $\rightarrow$ expression + term | term
term $\rightarrow$ term * factor | factor
factor $\rightarrow$ (expression) | N
 $N \rightarrow NZ \mid Z$
 $Z \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

Backus-Naur-Form

<expression> ::= **<expression> + <term> | <term>**
<term> ::= **<term> * <factor> | <factor>**
<factor> ::= **(<expression>) | <number>**
<number> ::= **<number> <digit> | <digit>**
<digit> ::= **0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9**

EXTENDED BACKUS-NAUR-FORM (EBNF)

Definition

- Literale in Anführungszeichen oder Apostroph
- = statt ::=
- Variablen ohne < und >, dürfen Leerzeichen enthalten
- Komma für Verkettung
- Regel-Endzeichen
- Kommentare (* ... *)
- Optionale Wiederholung { ... }
- Option [...]
- Gruppierung (...)
- Ausnahme: -

Achtung!

- Keine Operatorpriorisierung
- Zweideutigkeit!
- Keine eindeutige Evaluation!

Beispiel 11: EBNF für Expression-Term-Factor Grammatik

expression = expression, "+", term | term
term = term, "*", factor, | factor ;
factor = "(", expression, ")" | number ;
number = "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9" ;

RECHTSREKURSION

Expression-Term-Factor-Grammatik verwendet Links-Rekursion $\Rightarrow$ korrekte Auswertung von links nach rechts. Parsertree wertet aber Ausdrücke von rechts nach links aus! Alternative Expression-Term-Factor definition mit Rechtsrekursion statt Linksrekursion:

expression $\rightarrow$ term expression'
expression' $\rightarrow$ + term expression' | ε
term $\rightarrow$ factor term'
term' $\rightarrow$ * factor term' | ε
factor $\rightarrow$ (expression) | N
 $N \rightarrow NZ \mid Z$
 $Z \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

TODO:
grid $\rightarrow$ table

STACK
Unendlich grosser Speicher

- Immer nur das oberste Element sichtbar
- Beliebig tief

STACKAUTOMAT / PUSHDOWN AUTOMATON (PDA)
Kann Sprachen akzeptieren, die nicht regulär sind, ist aber nicht deterministisch. **Beachte:** $\Gamma \neq \Sigma$ möglich

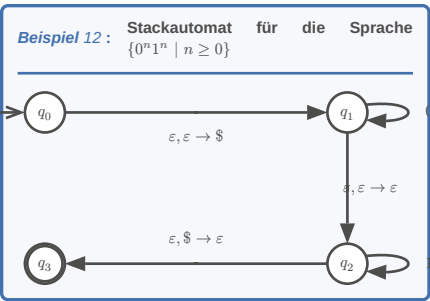
Definition
Stackautomat $(Q, \Sigma, \Gamma, \delta, q_0, F)$

Übergänge

$P =$

q vom Input
 b vom Stack entfernen (Bedingung)
 c auf den Stack

- Q : Zustände
- Σ : Eingabe-Alphabet
- Γ : Stack-Alphabet
- $\delta: Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \rightarrow P(Q \times \Gamma_\varepsilon)$
- $q_0 \in Q$: Startzustand
- $F \subset Q$: Akzeptierzustände

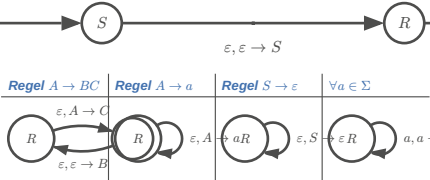


TODO:
Book 167-168, shift/reduce

TODO:
diagrams? (slides 8/12)

ABLEITUNG AUS CNF
Ist L eine kontextfreie Sprache, dann gibt es einen Stackautomaten P , der L akzeptiert, $L = L(P)$.

Grundgerüst



PDA $\rightarrow$ CFG
Variablen

Wörter beschreiben Pfade durch den Automaten: Variable A_{pq} = Wörter, die von p nach q führen mit leerem Stack

Regeln

Regeln beschreiben, wie sich Wege zerlegen lassen: $A_{pq} \rightarrow A_{pr} A_{rq}$ = Wege von p nach q können verstanden werden als Wege von p nach r und von dort nach q

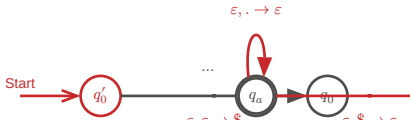
Stackautomat standardisieren

TODO:
better diagrams, book 181

- Nur ein Akzeptierzustand: Neuen Akzeptierzustand q_a und Übergänge von allen vorherigen Endzuständen zu q_a erstellen.



- Stack leeren: Mit einem Element $\$$ erweitern, sodass garantiert werden kann, dass der Stack am schluss leer ist.



- Jeder Übergang legt entweder ein Zeichen auf den Stack oder entfernt eines

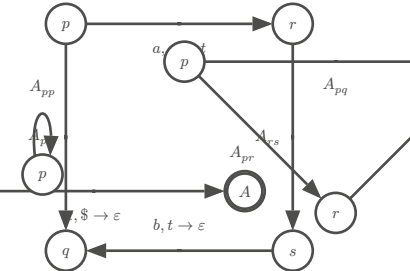


TODO:
Book 182,183

Beispiel 13
TODO:
visualization

(Grammatik ablesen)
Ausgangspunkt: standardisierter PDA mit Startzustand q_0 und $F = \{q_a\}$.

- Startvariable: $A_{q_0 q_a}$
- Regeln:



Beispiel 14
TODO:

NICHT KONTEXTFREIE SPRACHEN
PUMPING LEMMA

Pumping Lemma für CFL

Ist L eine CFL, dann gibt es eine Zahl N , die die Pumping Length, derart, dass jedes Wort $w \in L$ mit $|w| \geq N$ zerlegt werden kann in fünf Teile $w = uvxyz$ derart, dass

- $|vxy| > 0$
- $|vxy| \leq N$
- $uv^k xy^k z \in L \forall k \in \mathbb{N}$

Mit dem Pumping Lemma kann man beweisen, dass eine Sprache **nicht** kontextfrei ist.

Beispiel 15: $\{a^n b^n c^n \mid n \geq 0\}$

1) Annahme: L kontextfrei

2) Pumping length N

3) Wort: $w = a^N b^N c^N$

4) Zerlegungen:

5) Beim Pumpen nimmt die Anzahl der a und b zu, nicht aber die Anzahl der c
 $\Rightarrow uv^kxy^kz \notin L \forall k \neq 1$

6) Widerspruch: L nicht kontextfrei

Herleitung

Grammatik G in CNF

$$w \in L(G) \Rightarrow S \xrightarrow{*} w$$

Wiederverwendete Variable

$|w| \geq N$ gross genug $\Rightarrow$ Variablen werden im Parse Tree wiederverwendet Die «unterste» wiederverwendete Variable A erzeugt zwei Wörter:

$$A \Rightarrow vxy$$
$$A \Rightarrow x$$

Pumpen

$A \xrightarrow{*} vxy$ anstelle des «untersten» $A \xrightarrow{*} x$ verwenden

TODO:
diagram

Begriff	Bedeutung
Endlich	Eine Menge A heisst endlich , wenn jede Injektion $A \rightarrow A$ auch eine Bijektion ist
Abzählbar unendlich	Eine Menge A heisst abzählbar unendlich , wenn es eine Bijektion $\mathbb{N} \rightarrow A$ gibt, also $A \simeq \mathbb{N}$. Bsp: $\mathbb{R}, \mathbb{Z}, \mathbb{Q}, \Sigma^*$, Menge DEAs/NEAs/PDAs/CFGs
Überabzählbar unendlich	Eine Menge A heisst überabzählbar unendlich , wenn sie nicht abzählbar unendlich ist. Bsp: $\mathbb{C}, P(\mathbb{N})$, Menge aller Sprachen $P(\Sigma^*)$
Gleich mächtig	Mengen A und B heissen gleich mächtig , $A \simeq B$, wenn es eine Bijektion $A \rightarrow B$ gibt.
Unendlich	Eine Menge A heisst unendlich , wenn sie gleich mächtig wie eine Teilmenge ist

A, B, A_k abzählbar unendlich, $k \in \mathbb{N}$:

1) $A \cup B, \bigcup_k A_k$ abzählbar unendlich

2) $A \times B$ abzählbar unendlich

3) Abzählbar unendlich: $\mathbb{N}, \mathbb{Z}, \mathbb{Q}$

4) Der Inhalt der aktuellen Zelle kann beliebig verändert werden

5) $P(A)$ überabzählbar unendlich

6) $\mathbb{R}$ überabzählbar unendlich

TURING-MASCHINE (TM)

Merhere Stacks (Speicherzellen) = RAM (Band).

Jeder DEA oder NEA kann damit emuliert werden.

1) Der Speicher ist unbegrenzt

2) In jeder Zelle genau ein Zeichen

3) Es ist immer nur eine Speicherzelle einsehbar

4) Der Inhalt der aktuellen Zelle kann beliebig verändert werden

5) Bewegung immer nur eine Zelle nach links oder rechts

6) Kein weiterer Speicher (nur die Zustände eines endlichen Automaten und das eine Zeichen)

Begriff	Bedeutung
Record	Speichereinheit fester grösse
Bandalphabet	Alphabet Γ , welches aus Speicherzellen (Stacks) besteht
Schreib-/Lesekopf	Aktuelle Schreib-/Leseposition

Definition

(deterministische) Turing-Maschine

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$$

Q: Zustände

Σ : Alphabet

Γ : Bandalphabet, $\cup \in \Gamma \setminus \Sigma$

δ : $Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$

$q_0 \in Q$: Startzustand

$q_{\text{accept}} \in Q$: Akzeptierzustand

$q_{\text{reject}} \in Q$: Ablehnzustand, $q_{\text{accept}} \neq q_{\text{reject}}$

ARBEITSWEISE

Programmablauf

– Input-Wort w auf Band

– Schreib- / Lesekopf positioniert auf 1. Zeichen

– Maschine starten, $t(w)$ Einzelschritte ausführen

– Maschine hält in q_{accept} oder q_{reject} : Wort w akzeptiert / Verworfen

Laufzeit: $t(w), t(n) = \max\{t(w) \mid |w| \leq n\}$

Vorher

Nachher

Beispiel 16: $\{0^n 1^n \mid n \geq 0\}$

PROTOKOLLIERUNG

Der Gang der Berechnung mit einer TM kann dokumentiert werden, indem der Bandinhalt nach jedem einzelnen Verarbeitungsschritt protokolliert wird.

Links

Rechts

Beispiel 17: $L = \{0^n 1^n \mid n \geq 0\}, w = 0011$

VARIANTEN

Laufzeit: Die Laufzeit einer TM M bei der Verarbeitung eines Wortes $w \in \Sigma^*$ ist die Anzahl $t(w)$ der Schritte, die die Turing-Maschine ausführt, bis sie anhält.

NICHTDETERMINISTISCHE TM

Übergangsfunktion

Bei jedem Übergang maximal N verschiedene Möglichkeiten: $\delta : Q \times \Gamma \rightarrow P(Q \times \Gamma \times \{L, R\})$
 $\Rightarrow$ Mehrere mögliche Berechnungswege

Wort akzeptieren

$w \in L(M)$ wenn es einen Berechnungsweg gibt, der zu q_{accept} führt. (auch wenn es Wege gibt, die auf q_{reject} führen!)

Simulationsidee

Alle Berechnungswege durchführen, maximal $N^{t(n)}$

Simulierbar in $O(N^{t(n)}) = 2^{O(t(n))}$

Simulation auf Standard-TM

Verwende 3 Bänder:

1) Arbeitsband

2) Kopie von w

3) Liste aller Folgen von Wahlmöglichkeiten

Simulation

1) Kopiere w von Band 2 auf Band 1

2) Führe TM aus auf Band 1 mit Wahlmöglichkeiten von Band 3

3) Inkrementiere zur nächsten Folge von Wahlmöglichkeiten auf Band 3

VERSCHIEDENE BANDALPHABETE

Jedes andere Alphabet als $\Sigma = \{0, 1\}$ kann binär codiert werden.

MEHRSPURMASCHINE

TODO: buch ...250... slides 14

Simulierbar in Zeit $O(t(n))$

MEHRBANDMASCHINE

Um eine Mehrbandmaschine mit n Bändern auf einer einfacheren Maschine zu simulieren, kann die unabhängige Bewegung der Schreib-/Leseköpfe mithilfe eines $2n$ -spurigen Bandes (Daten+ Zeiger für Schreib-/Lesekopf) nachgebildet werden.

TODO: buch ...250... slides 14

Simulierbar in Zeit $O(t(n)^2)$

Harvard- und von Neumann-Architektur

Die AVR-Mikrokontroller-Familie von Atmel verwendet intern drei unabhängige Datenpfade:

– Flash-Speicher, der den Programmcode enthält

– Statisches RAM, welches zur Laufzeit veränderliche Daten speichert

– Peripheriegeräte.

Die von Neumann-Architektur vereinheitlicht Daten- und Programmspeicher.

EINSEITIG UNENDLICHES BAND

Beidseitig unendliches Band kann durch ein Alphabet doppelter Wortbreite realisiert werden, somit äquivalent.

TODO: buch 235

TURING-ERKENNBARE SPRACHEN

Eine TM erkennt das Wort $w \in \Sigma^*$, wenn die Maschine auf dem Input w im Zustand q_{accept} anhält.

Sind L_1 und L_2 Turing-erkennbare Sprachen, dann sind auch der Durchschnitt $L_1 \cap L_2$ und die Vereinigungsmenge $L_1 \cup L_2$ Turing-erkennbar.

Es gibt überabzählbare viele Sprachen $L \subset \Sigma^*$, die **nicht** Turing-erkennbar sind.

AUFZÄHLER

Aufzähler: TM mit einem Drucker, mit dem Wörter ausgedruckt werden können.

L ist **rekursiv aufzählbar** wenn es einen Aufzähler gibt, der alle Wörter aufzählen kann. L ist dann auch Turing-erkennbar.

ENTSCHEIDBAR UND ENTSCHEIDBARE SPRACHEN

Turing-erkennbare Sprache: L heisst **Turing-erkennbar**, wenn es eine TM M gibt mit $L = L(M)$.

Turing-entscheidbare Sprache: L heisst **Turing-entscheidbar**, wenn es einen Entscheider M gibt mit $L = L(M)$.

Berechenbare Funktion: Eine Funktion $f : \Sigma^* \rightarrow \Sigma^*$ heisst **berechenbar**, wenn es eine TM M gibt, die auf jedem Inputwort $w \in \Sigma^*$ anhält und auf dem Band das Outputwort $f(w)$ zurücklässt.

TODO:
nicht entscheidbare Probleme

ENTSCHEIDBARKEIT

«Die sich mit immer neuen Superlativen gegenseitig überbietenden Ankündigungen der IT-Industrie könnten den Eindruck erwecken, dass mit geeignet viel Rechenleistung und Speicherplatz kein Informatikproblem einer Lösung widerstehen könnte.»
— Prof. Dr. Andreas Müller, [1, p. 267]

based

TODO:
(Zehntens hilbertsches Problem). Man gebe ein Verfahren an, welches für eine beliebige diophantische Gleichung entscheidet, ob sie lösbar ist.

Serialisierung ermöglicht, jedes Entscheidungsproblem auf die Verarbeitung einer Zeichenkette zurückzuführen, die entweder akzeptiert oder verworfen wird. Jedes Problem wird auf diese Weise zu einem Sprachproblem.

ENTSCHEIDER

Definition

Beispiele

Ein **Entscheider** ist eine Turing-Maschine, die auf jedem beliebigen Input anhält.

$E_{\text{DEA}} = \{\langle A \rangle \mid A \text{ ein DEA und } L(A) = \emptyset\}$

Eine Sprache L heisst **entscheidbar**, wenn es einen Entscheider M gibt mit $L = L(M)$. Man sagt, M **entscheidet** L .

Gleichheitsproblem (Entscheidbar): $\{(G_1, G_2) \mid G_i \text{ CFGs und } L(G_1) = L(G_2)\}$

Akzeptanzproblem (Entscheidbar – nicht für TM!): $\{\langle A, w \rangle \mid A \text{ ein DEA, der } w \text{ akzeptiert}\}$

Halteproblem (Nicht entscheidbar): $\{\langle M, w \rangle \mid M \text{ hält auf Input } w\}$

AKZEPTANZPROBLEM FÜR TURING-MASCHINEN

$A_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ ist eine TM, die das Wort } w \in \Sigma^* \text{ akzeptiert}\}$

Theorem

A_{TM} ist nicht entscheidbar.

Beweis

Konstruiere aus einem Entscheider H für A_{TM} eine Maschine D mit Input $\langle M \rangle$

1) Lasse H auf Input $\langle M, \langle M \rangle \rangle$

2) Falls H akzeptiert: q_{reject}

3) Falls H verwirft: q_{accept}

Wende jetzt D auf $\langle D \rangle$ an:

$D(\langle D \rangle)$ akzeptiert $\Rightarrow D$ verwirft $\langle D \rangle$
 $D(\langle D \rangle)$ verwirft $\Rightarrow D$ akzeptiert $\langle D \rangle$

HALTEPROBLEM

$\text{HALT}_{\text{TM}} = \{\langle M, w \rangle \mid \text{die TM } M \text{ hält auf dem Input } w\}$

$\Rightarrow$ Gleicher Widerspruch wie beim Akzeptanzproblem. Umschreiben des Problems in ein neues Programm S , welches «Akzeptieren» in «Anhalten» übersetzen soll = Reduktion ($A_{\text{TM}} \leq \text{HALT}_{\text{TM}}$)

TODO:
spezielles/allgemeines Halteproblem

REDUKTION

Reduktionsabbildung

Berechenbare Abbildung $f : \Sigma^* \rightarrow \Sigma^*$ so, dass

$w \in A \Leftrightarrow f(w) \in B$

Notation: $f : A \leq B$ heisst « A leichter entscheidbar als B »

Entscheidbarkeit

Falls B entscheidbar, dann $f : A \leq B \Rightarrow A$ entscheidbar

Beweis

Ist H ein Entscheider für B , dann ist $H \circ f$ ein Entscheider für A .

Folgerung

A nicht entscheidbar, $A \leq B \Rightarrow B$ nicht entscheidbar

VERGLEICH DER ENTSCHEIDBARKEIT VON SPRACHEN

Beispiel 18

TODO:

SATZ VON RICE

LÖSUNGSMETHODEN FÜR ENTSCHEIDUNGSPROBLEME

TODO:
book 273..

KOMPLEXITÄT
Ineffizient: Entscheidbare Probleme mit exorbitant langer Laufzeit sind praktisch «nicht lösbar»
Problemgrösse: Laufzeit hängt von der Problemgrösse n ab
Worst case Laufzeit: Die worst case Laufzeit $t(n)$, die die Maschine zur Verarbeitung von Inputs der Länge n braucht, ist $t(n) = \max\{t(w) \mid |w| \leq n\}$

HARDWAREEINFLUSS
Nichtdeterministische Hardware kann in Zeit $2^{O(t(n))}$ simuliert werden $\Rightarrow$ NTM ist exponentiell schneller

POLYNOMIELLE UND EXPONENTIELLE LAUFZEIT

TODO:
slides 5

Polynomielle Probleme	Exponentielle Probleme
- Gauss-Algorithmus $O(n^3)$ - ...	- Faktorisierung von grossen Zahlen (RSA) - ...
Skalierbarkeit	Sicherheit

Ein Algorithmus hat **polynomielle Laufzeit**, wenn seine Laufzeit durch ein Polynom beschränkt ist: $t(n) = O(n^k)$.

SPRACHEN
Eine Sprache L gehört zur Klasse NP , wenn L von einer nichtdeterministischen TM in **polynomieller Zeit entschieden** werden kann

POLYNOMIELLE VERIFIZIERER
Ein **polynomieller Verifizierer** für die Sprache L ist eine TM V , so dass es für jedes $w \in \Sigma^*$ ein Wort c (das Lösungszertifikat) gibt, für das gilt

$w \in L \Leftrightarrow V$ akzeptiert $\langle w, c \rangle$

Die Laufzeit von V ist polynomiell in $|w|$.

Eine Sprache ist genau dann in **NP**, wenn sie in polynomieller Zeit **verifiziert** werden kann.

TODO:
slides 14

POLYNOMIELLE REDUKTION

POLYNOMIELLER LAUFZEIT-VERGLEICH

Seien A und B entscheidbare Sprachen. Eine berechenbare Abbildung

$f : \Sigma^* \rightarrow \Sigma^* : w \mapsto f(w)$

mit

1) $w \in A \Leftrightarrow f(w) \in B$ (Reduktion)
2) $f(w)$ ist berechenbar in polynomieller Zeit in $|w|$

heisst **polynomielle Reduktion** $A \leq_p B$. Lies: A ist polynomiell leichter entscheidbar als B .

Polynomiell äquivalent: Zwei Sprachen A und B heissen **polynomiell äquivalent**, geschrieben $A \equiv_p B$, wenn $A \leq_p B$ und $B \leq_p A$.

SATZ

Sei $A \leq_p B$. Dann gilt

$B \in P \Rightarrow A \in P, \quad A \notin P \Rightarrow B \notin P$
 $B \in NP \Rightarrow A \in NP, \quad A \notin NP \Rightarrow B \notin NP$

TODO:
slides 16,17

REDUKTION VON HAMPATH

TODO:

POLYNOMIELLE AUSFÜLLRÄTSEL

Ein polynomielles Ausfüllrätsel ist eine $n \times m$ -Tabelle, in die Zeichen eines Alphabets Σ eingefüllt werden müssen, so dass als logische Formel ausdrückbare Regeln eingehalten werden, die in polynomieller Zeit ausgewertet werden können. (Beispiel: Sudoku)

SAT: Boolean satisfiability problem: Gegeben einer aussagenlogischen Formel, gibt es eine Zusammensetzung der Variablenwerte, die die Aussage «Wahr» werden lassen?

$SAT = \{\varphi \mid \varphi \text{ ist eine erfüllbare logische Formel}\}$

Satz
Jedes polynomielle Ausfüllrätsel A lässt sich polynomiell auf SAT reduzieren.

Variablen
 $x_{ijc} = \text{wahr}$
 $\Leftrightarrow$ Feld (i, j) der Tabelle enthält Zeichen c

Genau ein Zeichen im Feld (i, j)

$$\varphi_{ij} = \bigvee_{c \in \Sigma} \left(\underbrace{x_{ijc} \wedge \bigvee_{d \neq c} x_{ijd}}_{c \text{ und kein anderes Zeichen in } (i, j)} \right)$$

Regeln
Spielregeln als Formel φ in Variablen x_{ijc} ausdrücken, die wahr ist genau dann, wenn die Regeln erfüllt sind.

NP-VOLLSTÄNDIGKEIT

Eine entscheidbare Sprache B heisst **NP-vollständig**, wenn sich jede Sprache A in NP polynomiell auf B reduzieren lässt:

$A \leq_p B \quad \forall A \in NP$

leichter schwieriger

NP-vollständige Probleme sind die «schwierigsten» Probleme in NP. Sie sind alle gleich schwierig:

A, B NP-vollständig $\Rightarrow (A \leq_p B \wedge B \leq_p A \Leftrightarrow A \equiv_p B)$
 $BIP = \{\langle C, d \rangle \mid C \in M_{n \times m}(\mathbb{Z}), d \in \mathbb{Z}^n, \text{ gibt es } x \in \{0, 1\}^m \text{ sodass } Cx = d\}$
 $CLIQUE-COVER = \{\langle G, k \rangle \mid\}$

Reduktionsprinzip
 A NP-vollständig $\wedge B \in NP \wedge A \leq_p B \Rightarrow B$ NP-vollständig

TODO:
slides 2

Wenn ein Problem NP-vollständig ist:
– Lösung braucht typischerweise exponentielle Zeit
– Korrektheit der Lösung ist in polynomieller Zeit zu prüfen

Umgang mit NP-Vollständigkeit in der Praxis:
– Spezialfälle ausnützen
– Problem modifizieren
– Approximation
– Zufall oder Heuristik
– Genetische Algorithmen und ML

Beispiele:
– SAT / 3SAT
– K-VERTEX-COLORING

COOK-LEVIN

3-SAT: Jede Klausel der Normalform hat maximal 3 Literale.

Satz
 SAT ist NP-vollständig.
Das bedeutet:
Sei A eine Sprache in NP
 $\Rightarrow$ Es gibt eine nichtdeterministische TM M , die A in polynomieller Zeit $O(t(n))$ entscheidet
 $\Rightarrow A$ kann polynomiell auf SAT reduziert werden: $A \leq_p SAT$

Vorgehensweise
Beschreibe das Finden der Berechnungsgeschichte von M als ein polynomielles Ausfüllrätsel

TODO:
slides 5

TODO:
slides 7

«Ausfüllrätsel» in diesem Fall die Berechnungsgeschichte der TM.

Überprüfe, ob alle Zustandsübergänge valid sind.

TODO:
book 328-343/368

SAT $\leq_p$ 3SAT

TODO:
 $\rightarrow$ Erfüllungsäquivalenz

Beweis von Cook-Levin
– Jedes Problem lässt sich auf eine Formel reduzieren
– Die Reduktion ist polynomiell
– Die Formel wird aus der Berechnungsgeschichte abgeleitet
– Die Formel kann in CNF konstruiert werden

Erfüllungsäquivalente Umformung

TODO:
slides 11

ÖFFENTLICHES SCHLÜSSELSYSTEM

TODO:
slides 15

KARP-KATALOG

TODO:
slides 16